

Bas — TD 1

SGF et commandes shell · Travaux dirigés · UE Bases de l'Architecture et des Systèmes · L1 Informatique, UPS · Semestre 1 2026-2027

Ce document reprend l'énoncé du TD1 (équipe pédagogique Système, UPS). **Partie 1** : les 10 énoncés seuls, pour s'entraîner sans être influencé par la réponse. **Partie 2** (après le saut de page) : le corrigé rédigé et vérifié de chaque exercice. Commandes de référence : `ls`, `echo`, `cat`, `grep`, `wc` (cf. CM).

Partie 1 — Énoncés

1. Syntaxe de description d'une commande Unix

Exercice 1

La syntaxe de `cut` est décrite par : `cut -c list [file ...]` ou `cut -f list [-d delim] [-s] [file ...]`. Les appels suivants sont-ils corrects ?

- a) `cut -d : -f 1,7 fichier`
- b) `cut -c 1-16,26-38`
- c) `cut -s fich1 fich2`
- d) `cut -f 2 -c 1,3,5 fichier`

Exercice 2

Commande `tr` à 3 options `f`, `c`, `d` : `c` et `d` facultatives mais exclusives entre elles, `f` obligatoire ; appel avec une liste **non vide** de fichiers. Écrire la syntaxe d'appel.

2. Désignation d'un fichier

Exercice 3

Dans une désignation, tous les caractères `/` ont-ils la même signification ?

Exercice 4

1) La désignation relative et la désignation absolue d'un fichier sont-elles uniques ? 2) Laquelle est la plus courte (en nombre de répertoires) pour désigner un même élément ?

Exercice 5

Répertoire courant : `/users/linfg/l2inf201/REP1`. Donner la désignation relative du fichier `/users/linfg/l2inf201/REP1/f1`.

3. Métacaractères du shell

Exercice 6

- 1) Décrire le modèle `REP/*.*??`. 2) Écrire, avec les métacaractères du shell, les modèles décrivant :
- a) le caractère `a` ou `b` ou `c` ou `d` ou `e`
 - b) le caractère `a` suivi d'un chiffre ou du caractère `b`
 - c) une minuscule ou une majuscule
 - d) une minuscule suivie d'un chiffre

Exercice 7

On suppose qu'il n'y a pas de sous-répertoire dans le répertoire courant. Écrire, en une seule commande à chaque fois :

- a) afficher les noms de fichiers non cachés dont le nom contient le caractère a
- b) afficher la liste détaillée des fichiers non cachés dont le nom comporte 4 caractères exactement
- c) afficher le contenu des fichiers non cachés dont le nom comporte au moins 5 caractères
- d) afficher le nombre de lignes de chacun des fichiers non cachés dont le nom commence par une majuscule

Exercice 8

Les répertoires REP1 et REP1/SREP1 sont dans le répertoire courant. 1) Avec `ls` une seule fois, afficher le contenu de REP1 et REP1/SREP1. 2) Même question avec `echo`. 3) Même résultat ?

4. Redirections

Exercice 9

1) Donner trois manières différentes d'afficher les lignes du fichier `truc.txt` contenant au moins un e. 2) Qu'est-ce qui change lors de l'exécution ?

Exercice 10

1) Commande qui enregistre dans `liste.txt` les noms de tous les fichiers et sous-répertoires du répertoire courant. 2) Commande affichant les lignes des fichiers dont les noms sont dans `liste.txt` et qui contiennent au moins un e.

Partie 2 — Corrigé

1. Syntaxe de description d'une commande Unix

Exercice 1

Appel	Réponse
<code>cut -d : -f 1,7 fichier</code>	Correct. Utilise la 2e syntaxe : <code>-f 1, 7</code> (list), <code>-d :</code> (délimiteur), <code>fichier</code> . L'ordre des options n'a pas à respecter l'ordre de la description (les crochets et l'absence de crochets décrivent ce qui est obligatoire/facultatif, pas un ordre à la frappe).
<code>cut -c 1-16,26-38</code>	Correct. Utilise la 1re syntaxe : <code>-c</code> avec <code>list=1-16,26-38</code> . Le fichier est facultatif (<code>[file ...]</code>), donc son absence est valide (lecture sur <code>stdin</code>).
<code>cut -s fich1 fich2</code>	Incorrect. <code>-s</code> n'existe que dans la 2e syntaxe, qui impose <code>-f list</code> en premier (non entre crochets = obligatoire). Ici <code>-f list</code> est absent.
<code>cut -f 2 -c 1,3,5 fichier</code>	Incorrect. <code>-f</code> et <code>-c</code> appartiennent à deux syntaxes distinctes et exclusives (deux façons alternatives d'appeler <code>cut</code>) : on ne peut pas les combiner dans le même appel.

Exercice 2

Réponse :

```
truc -f [-c|-d] file [file ...]
```

`-f` est écrit sans crochets (obligatoire). `[-c|-d]` : crochets = facultatif, `|` = choix exclusif entre les deux. `file [file ...]` : un premier fichier obligatoire (liste non vide), suivi éventuellement d'autres fichiers (répétition facultative).

2. Désignation d'un fichier

Exercice 3

Non. La plupart des `/` jouent le rôle de **séparateurs** entre noms de répertoires successifs. Mais le `/` placé tout au début d'une désignation **absolue** a un sens différent : il désigne la **racine** elle-même de l'arborescence, pas un simple séparateur.

Exercice 4

- **Désignation absolue : unique** (à formulation canonique) — l'arborescence étant un arbre, il n'existe qu'un seul chemin depuis la racine jusqu'à un fichier donné.
- **Désignation relative : pas unique** — elle dépend du répertoire courant, qui peut changer ; et même pour un répertoire courant fixé, plusieurs écritures différentes (utilisant `./`...) peuvent désigner le même fichier.
- **La plus courte** : la désignation **relative** est en générale plus courte (ou égale) que l'absolue, car elle part du répertoire courant au lieu de la racine — l'absolue doit en plus traverser tous les répertoires intermédiaires entre la racine et le répertoire courant.

Exercice 5

```
f1
```

Le fichier `f1` est directement dans le répertoire courant `REP1` : sa désignation relative est donc simplement son nom, `f1` (pas besoin de préfixer par `./`, bien que ce soit équivalent).

3. Métacaractères du shell

Exercice 6

1) `REP/* ???` décrit les fichiers du répertoire `REP` dont le nom se termine par un point suivi d'exactly **3 caractères** (une extension de 3 caractères quelconques) — par exemple `toto.txt` ou `a.png`, mais pas `a.c` (extension d'1 caractère) ni `a.html` (4 caractères).

Description	Modèle
le caractère a ou b ou c ou d ou e	[abcde] (ou [a-e])
le caractère a suivi d'un chiffre ou du caractère b	a[0-9b]
une minuscule ou une majuscule	[a-zA-Z]
une minuscule suivie d'un chiffre	[a-z][0-9]

Pour `a[0-9b]` : à lire comme « a, suivi d'un chiffre ou de b » (et non « a suivi d'un chiffre, OU simplement b seul », qui ne pourrait pas s'écrire comme un modèle unique de longueur fixe avec `*?/[ ]`). Vérifié : matche `a1`, `ab`, `a9` mais pas `az` ni `bb`.

Exercice 7

Demande	Commande
Afficher les noms de fichiers non cachés dont le nom contient le caractère a	<code>ls *a*</code>
Afficher la liste détaillée des fichiers non cachés dont le nom comporte 4 caractères exactement	<code>ls -l ????</code>
Afficher le contenu des fichiers non cachés dont le nom comporte au moins 5 caractères	<code>cat ?????*</code>
Afficher le nombre de lignes de chacun des fichiers non cachés dont le nom commence par une majuscule	<code>wc -l [A-Z]*</code>

Dans chaque cas, l'exclusion des fichiers cachés est **automatique** : `*` et `?` ne correspondent pas à un `.` initial quand ils sont en début de modèle, et `[A-Z]` ne contient de toute façon pas le caractère `.`. Toutes ces commandes ont été testées (jeu de fichiers de test) et matchent exactement l'ensemble attendu.

Exercice 8

```
ls REP1 REP1/SREP1
```

→ affiche, avec un en-tête par répertoire :

```
REP1:
SREP1
f1
f2

REP1/SREP1:
g1
g2
```

```
echo REP1/* REP1/SREP1/*
```

→ affiche (une fois les métacaractères interprétés par le shell) :

```
REP1/SREP1 REP1/f1 REP1/f2 REP1/SREP1/g1 REP1/SREP1/g2
```

3) Non, pas le même résultat. Les deux solutions listent globalement les mêmes fichiers, mais pas de la même façon : `ls` affiche chaque répertoire séparément, précédé de son nom en en-tête ; `echo` affiche tous les noms (avec leur chemin complet, puisqu'ils viennent du remplacement de `REP1/*` et `REP1/SREP1/*`) à la suite les uns des autres, sur une **seule ligne**, sans aucune distinction entre les deux répertoires d'origine.

4. Redirections

Exercice 9

```
grep e truc.txt # grep ouvre lui-meme le fichier
grep e < truc.txt # le shell connecte stdin de grep au fichier
cat truc.txt | grep e # cat lit le fichier, le pipe vers grep
```

2) Le résultat affiché à l'écran est **identique** dans les trois cas. Ce qui change, c'est le mécanisme : dans le 1er cas, **grep** ouvre lui-même le fichier passé en paramètre ; dans le 2e, c'est le **shell** qui connecte l'entrée standard de **grep** au fichier (**grep** ne sait pas qu'il lit un fichier particulier, il lit **stdin**) ; dans le 3e, un **processus supplémentaire** (**cat**) est lancé, qui lit le fichier et écrit son contenu sur **stdout**, connecté par le pipe à l'entrée standard de **grep** — donc **deux processus** au lieu d'un seul.

Exercice 10

```
ls > liste.txt
```

ls affiche un nom par ligne (fichiers et sous-répertoires), la redirection **>** écrit ce résultat dans **liste.txt**.

```
grep e `cat liste.txt` # ou : grep e $(cat liste.txt)
```

La capture ``cat liste.txt`` transforme le contenu de **liste.txt** (un nom de fichier par ligne) en une liste de **paramètres** pour **grep** (sauts de ligne remplacés par des espaces) — exactement le mécanisme de capture vu en cours (`grep 'e' `ls``).

À retenir

- **Syntaxe de commande** : pas de crochets = obligatoire, `[...]` = facultatif, `...` = répétable, `|` = choix exclusif ; deux lignes de syntaxe séparées = deux formes **exclusives**, on ne mélange pas leurs options.
- **Désignations** : l'absolue (depuis la racine `/`) est unique mais plus longue ; la relative (depuis le répertoire courant) est plus courte mais dépend du contexte, donc pas unique.
- **Métacaractères** : `*/?` excluent automatiquement les fichiers cachés en début de modèle ; `[a-zA-Z]`, `[0-9]` pour des classes de caractères ; combiner plusieurs `[ ]/?` pour des positions successives (ex. `a[0-9b]`, `[a-z][0-9]`).
- **ls vs echo sur un glob** : **ls** liste un nom par ligne avec en-tête par répertoire ; **echo** écrit tout sur une seule ligne, sans distinction — mêmes fichiers, présentation différente.
- **Redirections** : fichier en paramètre, `<` ou pipe donnent le même résultat affiché mais pas le même nombre de processus lancés. La capture ``cat fichier`` transforme le contenu d'un fichier de noms en liste de paramètres pour une autre commande.